

Contents

What is Gradle (in simple terms)?	3
1. Typical Java Gradle Project Structure	3
2. Core Gradle Files (VERY IMPORTANT).....	4
build.gradle.....	4
settings.gradle.....	5
3. Gradle Wrapper (Enterprise-Level Concept).....	6
gradlew / gradlew.bat	6
gradle/wrapper/gradle-wrapper.properties.....	6
4. Source Code Structure (src folder)	7
src/main/java	7
src/main/resources.....	7
src/test/java	7
src/test/resources.....	7
5. build/ Directory (Auto-generated)	8
6. Common Gradle Commands (You MUST know)	8
7. Gradle Concepts You'll Hear at Work.....	9
Plugins	9
Dependencies.....	9
Tasks.....	9
Crete Gradle Project	10
STEP 1: Create a New Gradle Project in IntelliJ.....	10
STEP 2: Project Details	10
STEP 3: Gradle Settings (IMPORTANT)	10
STEP 4: Let IntelliJ Build the Project.....	11
STEP 5: Understand the Generated Project Structure.....	11
STEP 6: Configure build.gradle.....	12
STEP 7: Create Your First Java Program	12
STEP 8: Run the Application (2 Ways)	13
STEP 9: Run Tests	13
STEP 10: Explore Gradle Tasks in IntelliJ	13
Understanding Tasks	14
1. What is a Gradle Task? (Plain English)	14
2 Why Tasks Exist in a Java Project	14
3 Tasks You Already Used (Without Knowing)	15

4 Core Tasks in a Java Gradle Project (Must Know)	15
5 Why Tasks Are Better Than Shell Scripts.....	15
6 How Tasks Are Used in Real Java Projects	16
7 Task Dependencies (VERY IMPORTANT CONCEPT).....	16
8 Types of Tasks (Beginner Friendly).....	16
9 Create Your Own Task (Hands-on)	16
10. Task Phases (CRITICAL INTERVIEW TOPIC).....	17
11 Task Configuration vs Execution (Common Mistake)	17
12 How IntelliJ Shows Tasks.....	17
13 Tasks – Solve Tasks (Answers last)	18
Tasks – Practice questions (Answers last).....	27
Tasks - Interview questions.....	34
BASIC QUESTIONS (Foundations).....	34
INTERMEDIATE QUESTIONS	35
ADVANCED / IMPRESSION QUESTIONS	36
More things to Learn.....	37

What is Gradle (in simple terms)?

Gradle is a **build automation tool** used to:

- Compile Java code
- Download dependencies (JARs)
- Run tests
- Package applications (JAR/WAR)
- Automate tasks (build, clean, test, run)

Think of Gradle as:

“The manager that knows how to build, test, and package your Java project.”

1. Typical Java Gradle Project Structure

```
my-gradle-project/
├── build.gradle
├── settings.gradle
├── gradlew
├── gradlew.bat
├── gradle/
│   └── wrapper/
│       ├── gradle-wrapper.properties
│       └── gradle-wrapper.jar
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/app/
│   │   │       └── App.java
│   │   └── resources/
│   │       └── application.properties
│   └── test/
│       ├── java/
│       │   └── com/example/app/
│       │       └── AppTest.java
│       └── resources/
└── build/
```

2. Core Gradle Files (VERY IMPORTANT)

build.gradle

This is the **heart of the project**. What it does:

- Defines **plugins**
- Defines **dependencies**
- Defines **Java version**
- Defines **tasks**

Example:

```
plugins {  
    id 'java'  
}  
  
group = 'com.example'  
version = '1.0.0'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(17)  
    }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    implementation 'org.apache.commons:commons-lang3:3.14.0'  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'  
}  
  
test {  
    useJUnitPlatform()  
}
```

Key blocks explained:

Block	Purpose
plugins	Adds Java support
group	Base package name
version	Project version
repositories	Where dependencies come from
dependencies	Libraries your code uses
test	Test configuration

Defines the **project name** and **modules**.

Example:

```
rootProject.name = 'my-gradle-project'
```

For multi-module projects:

```
include 'service', 'common'
```

3. Gradle Wrapper (Enterprise-Level Concept)

`gradlew / gradlew.bat`

Used to run Gradle **without installing Gradle globally**.

Run commands like:

```
./gradlew build      # Linux/Mac
gradlew build        # Windows
```

✓ Ensures **same Gradle version** for all developers.

`gradle/wrapper/gradle-wrapper.properties`

Defines **which Gradle version** to use.

```
distributionUrl=https\://services.gradle.org/distributions/gradle-8.6-bin.zip
```

4. Source Code Structure (`src` folder)

`src/main/java`

This is where **production code** lives.

```
src/main/java/com/example/app/App.java
```

Package name should match folder structure:

```
package com.example.app;
```

`src/main/resources`

Used for:

- Configuration files
- Properties
- XML / YAML
- Static files

Example:

```
application.properties  
logback.xml
```

These files go **inside the final JAR**.

`src/test/java`

Contains **unit and integration tests**.

Example:

```
class AppTest {  
    @Test  
    void testApp() {  
        assertTrue(true);  
    }  
}
```

`src/test/resources`

Test-specific configuration:

```
application-test.properties
```

5. build/ Directory (Auto-generated)

Created by Gradle after running build commands.

Contains:

- Compiled `.class` files
- Test reports
- JAR/WAR files

⚠️ Never edit or commit this folder.

6. Common Gradle Commands (You MUST know)

Command	Purpose
<code>gradlew build</code>	Compile + test + package
<code>gradlew clean</code>	Delete build folder
<code>gradlew test</code>	Run tests
<code>gradlew tasks</code>	List all tasks
<code>gradlew dependencies</code>	View dependency tree
<code>gradlew bootRun</code>	Run Spring Boot app

7. Gradle Concepts You'll Hear at Work

Plugins

Extend Gradle's capability:

```
id 'java'  
id 'application'  
id 'org.springframework.boot'
```

Dependencies

Type	Meaning
implementation	Used at runtime
compileOnly	Needed only at compile
runtimeOnly	Needed only at runtime
testImplementation	For tests

Tasks

Gradle executes **tasks**.

Example:

```
task hello {  
    doLast {  
        println 'Hello Gradle'  
    }  
}
```

Run:

```
gradlew hello
```

Ceate Gradle Project

STEP 1: Create a New Gradle Project in IntelliJ

1. Open **IntelliJ IDEA**
2. Click **New Project**
3. On the left panel, select **Gradle**
4. On the right:
 - o ☒ Select **Java** ☒ Do NOT select Kotlin/Groovy
5. Click **Next**

STEP 2: Project Details

Fill the details like this (example):

Field	Value
Name	hello-gradle
Location	Choose any folder
GroupId	com.chandra
ArtifactId	hello-gradle

Click **Next**.

STEP 3: Gradle Settings (IMPORTANT)

You'll see Gradle options:

Option	Select
Gradle DSL	Groovy (recommended for beginners)
Use Gradle wrapper	☒ YES
Gradle JVM	Java 25
Distribution	Wrapper

Why Wrapper?

It ensures the same Gradle version for everyone (industry standard).

Click **Create**.

STEP 4: Let IntelliJ Build the Project

- IntelliJ will download Gradle
- You'll see **"Building Gradle project"**
- Wait until it finishes (important)

STEP 5: Understand the Generated Project Structure

Your project will look like this:

```
hello-gradle/
├── build.gradle
├── settings.gradle
├── gradlew
├── gradlew.bat
├── gradle/
│   └── wrapper/
└── src/
    ├── main/
    │   └── java/
    │       └── com/chandra/
    │           └── App.java
    └── test/
        └── java/
            └── com/chandra/
                └── AppTest.java
```

Now let's **edit and understand each important file.**

STEP 6: Configure `build.gradle`

Open **build.gradle** and replace everything with this:

```
plugins {  
    id 'java'  
    id 'application'  
}  
  
group = 'com.chandra'  
version = '1.0.0'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(25)  
    }  
}  
  
repositories {  
    mavenCentral()  
}  
  
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'  
}  
  
application {  
    mainClass = 'com.chandra.App'  
}  
  
test {  
    useJUnitPlatform()  
}
```

This tells Gradle:

- Use **Java**
- Use **Java 25**
- Where to download libraries
- Which class is the **main entry point**

STEP 7: Create Your First Java Program

Open: `src/main/java/com/chandra/App.java`

Replace content with:

```
package com.chandra;  
  
public class App {  
    public static void main(String[] args) {  
        System.out.println("Hello Gradle! ☐");  
    }  
}
```

STEP 8: Run the Application (2 Ways)

✓ Option 1: Using IntelliJ (Beginner Friendly)

1. Right-click `App.java`
2. Click **Run 'App.main()'**
3. Output:

```
Hello Gradle!
```

✓ Option 2: Using Gradle (Industry Standard)

Open **Terminal** inside IntelliJ and run:

```
gradlew run
```

Output:

```
Hello Gradle!
```

✓ This is how CI/CD tools (Jenkins) run your app.

STEP 9: Run Tests

```
gradlew test
```

Test results go to:

```
build/reports/tests/test/index.html
```

STEP 10: Explore Gradle Tasks in IntelliJ

On the **right side**, open:

```
Gradle → Tasks → application → run
```

Double-click **run** → app runs ☐

Understanding Tasks

1. What is a Gradle Task? (Plain English)

A **Gradle task** is a **single unit of work**. Think of a task as:

“One specific action Gradle knows how to perform.”

Examples of actions:

- Compile Java code
- Run tests
- Create a JAR file
- Clean old files
- Run your application

Each of these actions is a **task**.

2 Why Tasks Exist in a Java Project

In a Java project, many steps are needed:

1. Compile `.java` → `.class`
2. Copy resources
3. Run tests
4. Package everything into a JAR
5. Run the app

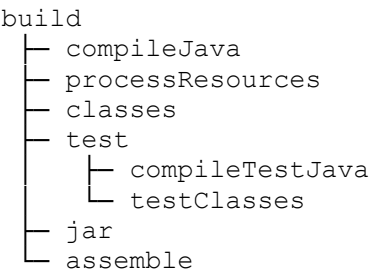
Gradle breaks these steps into **small reusable tasks** so that:

- ✓ Builds are **fast** (only run what changed)
- ✓ Steps are **repeatable**
- ✓ CI tools can run them
- ✓ Developers don't do things manually

3 Tasks You Already Used (Without Knowing)

When you ran: `gradlew build`

You **did NOT run one task**. You ran a **task graph** like this:



build is just a task that depends on other tasks.

4 Core Tasks in a Java Gradle Project (Must Know)

Task	What it does
<code>clean</code>	Deletes the <code>build/</code> folder
<code>compileJava</code>	Compiles Java source
<code>processResources</code>	Copies config files
<code>classes</code>	Combines compiled output
<code>test</code>	Runs JUnit tests
<code>jar</code>	Creates a JAR
<code>build</code>	Runs everything needed

5 Why Tasks Are Better Than Shell Scripts

Imagine you did this manually:

```
javac App.java
java App
```

Problems	Gradle tasks solve this
✗ Not repeatable	✓ Automatic order
✗ No dependency handling	✓ Automatic dependency download
✗ No caching	✓ Incremental builds
✗ Hard in teams	✓ Cross-platform

:

:

6 How Tasks Are Used in Real Java Projects

Example: CI/CD (Jenkins)

```
gradlew clean build
```

Example: Run application

```
gradlew run
```

Example: Run only tests

```
gradlew test
```

Each command is a **task or group of tasks**.

7 Task Dependencies (VERY IMPORTANT CONCEPT)

Tasks can depend on other tasks.

Example:

```
tasks.named('build') {  
    dependsOn 'test'  
}
```

Meaning: “Before `build` runs, make sure `test` runs.”

Gradle **automatically decides the order**.

8 Types of Tasks (Beginner Friendly)

Lifecycle Tasks	Action Tasks
High-level tasks that don't do work directly. Examples: • <code>build</code> • <code>assemble</code> • <code>check</code>	Do actual work. Examples: • <code>compileJava</code> • <code>test</code> • <code>jar</code>

9 Create Your Own Task (Hands-on)

Add this to `build.gradle`:

```
tasks.register('helloTask') {  
    doLast {  
        println "Hello from my first Gradle task!"  
    }  
}
```

Run: `gradlew helloTask`

Output: `Hello from my first Gradle task!`

You just created a custom task.

10. Task Phases (CRITICAL INTERVIEW TOPIC)

Gradle runs in **3 phases**:

1 Initialization

- Reads `settings.gradle`
- Decides which projects to build

2 Configuration

- Reads `build.gradle`
- Configures all tasks

3 Execution

- Executes only required tasks

Tasks in `doLast {}` run in **execution phase**.

11 Task Configuration vs Execution (Common Mistake)

✗ Wrong:

```
tasks.register('badTask') {  
    println "Runs always!"  
}
```

✓ Correct:

```
tasks.register('goodTask') {  
    doLast {  
        println "Runs only when task is executed"  
    }  
}
```

12 How IntelliJ Shows Tasks

In IntelliJ:

Gradle → Tasks → build → build

Double-click = run task.

This is **exactly what Jenkins does**.

13 Tasks – Solve Tasks (Answers last)

Q1

```
tasks.register('taskA') {  
    doLast {  
        println "Task A executed"  
    }  
}
```

Question:

What command will execute this task? What will be printed?

Q2

```
tasks.register('taskB') {  
    println "Configuring Task B"  
    doLast {  
        println "Executing Task B"  
    }  
}
```

Question:

When you run:

```
gradlew taskB
```

Which lines print during **configuration** and which during **execution**?

Q3

```
tasks.register('taskC') {  
    doLast {  
        println "Hello Gradle"  
    }  
}
```

Question:

What happens if you run:

```
gradlew tasks
```

Will "Hello Gradle" print or not? Why?

Q4

```
tasks.register('task1') {
    doLast {
        println "Task 1"
    }
}

tasks.register('task2') {
    dependsOn 'task1'
    doLast {
        println "Task 2"
    }
}
```

Question: What is the output when you run: `gradlew task2`

Q5

```
tasks.register('taskX') {
    doLast {
        println "Task X"
    }
}

tasks.register('taskY') {
    mustRunAfter 'taskX'
    doLast {
        println "Task Y"
    }
}
```

Question: What happens if you run: `gradlew taskY`

Will `taskX` run or not?

Q6

```
tasks.register('cleanData') {
    doLast {
        println "Cleaning data"
    }
}

tasks.register('buildData') {
    dependsOn 'cleanData'
    doLast {
        println "Building data"
    }
}
```

Question:

Why is `dependsOn` preferred here instead of `mustRunAfter`?

Q7

```
tasks.register('taskP') {
    println "Config P"
    doLast {
        println "Run P"
    }
}

tasks.register('taskQ') {
    println "Config Q"
    dependsOn 'taskP'
    doLast {
        println "Run Q"
    }
}
```

Question:

When running: `gradlew taskQ`

What is the **exact order of all printed lines**?

Q8

```
tasks.register('taskR') {
    doLast {
        println "R"
    }
}

tasks.register('taskS') {
    dependsOn 'taskR'
}

tasks.register('taskT') {
    dependsOn 'taskR'
}
```

Question: When running: `gradlew taskS taskT`

How many times does `taskR` execute? Why?

Q9

```
tasks.register('taskOnly') {
    doLast {
        println "I run only if asked"
    }
}
```

Question:

What happens when you run: `gradlew build`

Does `taskOnly` execute? Why or why not?

Q10

```
tasks.register('taskConfig') {  
    println "Configuration logic"  
}
```

Question: Why is this task considered **bad practice**?

Q11

```
tasks.register('taskExec') {  
    doFirst {  
        println "First"  
    }  
    doLast {  
        println "Last"  
    }  
}
```

Question: What is the output and execution order?

Q12

```
tasks.register('taskA') {  
    doLast {  
        println "A"  
    }  
}  
  
tasks.register('taskB') {  
    dependsOn 'taskA'  
    doLast {  
        println "B"  
    }  
}
```

Question: What changes if you replace `dependsOn 'taskA'` with: `mustRunAfter 'taskA'`

Q13

```
tasks.register('taskSkip') {  
    onlyIf {  
        false  
    }  
    doLast {  
        println "Should not run"  
    }  
}
```

Question: What happens when you run:

```
gradlew taskSkip
```

Q14

```
tasks.register('taskInput') {
    inputs.property("name", "Gradle")
    outputs.file("$buildDir/output.txt")

    doLast {
        println "Processing input"
    }
}
```

Question:

Why would Gradle skip this task on second run?

Q15 (Hard but important)

```
tasks.register('taskDynamic') {
    doLast {
        tasks.register('innerTask') {
            doLast {
                println "Inner Task"
            }
        }
    }
}
```

Question:

Will `innerTask` execute in the same build run? Why?

Answers

Q1 Answer

Command: `gradlew taskA`

Output: Task A executed

Why: The task has only a `doLast` block, which runs during execution.

Q2 Answer

```
tasks.register('taskB') {  
    println "Configuring Task B"  
    doLast {  
        println "Executing Task B"  
    }  
}
```

Output when running `gradlew taskB`:

```
Configuring Task B      ← configuration phase  
Executing Task B        ← execution phase
```

Why: Code outside `doLast` runs during **configuration**, even if the task is not executed.

Q3 Answer

`gradlew tasks`

Output:

✗ "Hello Gradle" is NOT printed

Why: `doLast` runs only when the task is executed. `gradlew tasks` only lists tasks.

Q4 Answer

Command: `gradlew task2`

Output:

```
Task 1  
Task 2
```

Why: `task2` depends on `task1`, so Gradle runs `task1` first.

Q5 Answer

```
gradlew taskY
```

Output: Task Y

Why:

`mustRunAfter` does **NOT** create a dependency.

`taskX` runs **only if explicitly requested**.

Q6 Answer

Why `dependsOn` is preferred:

Because `buildData` **cannot run correctly without `cleanData`**.

`dependsOn` ensures execution, while `mustRunAfter` only controls order.

Q7 Answer

Command: `gradlew taskQ`

Output order:

```
Config P
```

```
Config Q
```

```
Run P
```

```
Run Q
```

Why:

- All tasks are configured first
- Then execution happens respecting dependencies

Q8 Answer

```
gradlew taskS taskT
```

`taskR` executes:

✓ **Only once**

Why:

Gradle executes shared dependencies only once in a task graph.

Q9 Answer

```
gradlew build
```

Does taskOnly run?

✗ No

Why:

Tasks execute only if:

- Explicitly requested, or
- Required as a dependency

Q10 Answer

```
tasks.register('taskConfig') {  
    println "Configuration logic"  
}
```

Why this is bad practice:

Because it runs during **configuration phase**, even if the task is never executed.
This slows builds and breaks task avoidance.

✓ Fix:

```
tasks.register('taskConfig') {  
    doLast {  
        println "Execution logic"  
    }  
}
```

Q11 Answer

```
gradlew taskExec
```

Output:

```
First  
Last
```

Why:

- doFirst runs before task actions
- doLast runs after

Q12 Answer

With `dependsOn`:

A
B

With `mustRunAfter`:

B

Why: `mustRunAfter` does not trigger execution of `taskA`.

Q13 Answer

```
gradlew taskSkip
```

Output:

✗ Nothing printed

Gradle output:

```
Task :taskSkip SKIPPED
```

Why: `onlyIf { false }` prevents execution.

Q14 Answer

Why Gradle skips the task on second run:

Because:

- Inputs didn't change
- Outputs already exist

Gradle marks task as: UP-TO-DATE

Q15 Answer (Hard & Important)

Will `innerTask` run? ✗ NO

Why:

- Tasks are resolved during **configuration phase**
- `innerTask` is created during **execution phase**
- Gradle cannot add tasks mid-execution

✓ Correct approach:

Define all tasks during configuration.

14. Tasks – Practice questions (Answers last)

Q1 Write a task named `helloTask` that prints:

```
Hello from Gradle
```

Run only this task.

Q2 Write a task named `javaInfo` that prints:

- Java version
- Java home

(Use Java system properties.)

Q3 Write two tasks:

- `taskOne` → prints "Task One"
- `taskTwo` → prints "Task Two"

Run **only** `taskTwo`.

Q4 Create two tasks:

- `cleanTemp`
- `buildTemp`

Ensure:

- `cleanTemp` **always runs before** `buildTemp`
- `buildTemp` **cannot run without** `cleanTemp`

Q5 Create two independent tasks:

- `alpha`
- `beta`

Ensure:

- `beta` runs **after** `alpha`
- Running `beta` **alone** should NOT execute `alpha`

Q6 Create:

- `prepare`
- `compile`
- `packageApp`

Ensure execution order: `prepare` → `compile` → `packageApp`

Run **only** `packageApp`.

Q7 Write a task `conditionalTask` that:

- Executes **only if** a project property named `runTask` is set
- Prints `"Task Executed"`

Run it once **with** and once **without** the property.

Q8 Write a task `onceOnly` that:

- Prints `"Executed"`
- Is skipped automatically if nothing has changed (UP-TO-DATE behavior)

Q9 Write two tasks:

- `parentTask`
- `childTask`

Ensure:

- `childTask` runs **before** `parentTask`
- `parentTask` explicitly depends on `childTask`

Q10 Write a task `configVsExec` that prints:

- `"Config phase"` during configuration
- `"Execution phase"` only when the task runs

Q11 Write a task `dualAction` that prints:

Start
End

Using **both** `doFirst` and `doLast`.

Q12 Write a task `fileCreator` that:

- Creates a file inside `build/`
- Uses proper **inputs & outputs**
- Becomes UP-TO-DATE on second run

Q13 Write a task `skipOnCI` that:

- Skips execution if environment variable `CI=true`
- Prints "Running locally" otherwise

Q14 Write two tasks:

- `taskA`
- `taskB`

Ensure:

- Both depend on a common task `setup`
- `setup` executes **only once**

Q15 (Hard)

Write a task `safeTask` that:

- Does NOT execute if another task named `dangerTask` is in the task graph
- Prints "Safe execution" otherwise

Answers

Answer for Q1 – helloTask

```
tasks.register('helloTask') {  
    doLast {  
        println "Hello from Gradle"  
    }  
}
```

Run:

```
gradlew helloTask
```

Answer for Q2 – javaInfo

```
tasks.register('javaInfo') {  
    doLast {  
        println "Java Version: ${System.getProperty('java.version')}}"  
        println "Java Home: ${System.getProperty('java.home')}}"  
    }  
}
```

Answer for Q3 – Run only taskTwo

```
tasks.register('taskOne') {  
    doLast {  
        println "Task One"  
    }  
}  
  
tasks.register('taskTwo') {  
    doLast {  
        println "Task Two"  
    }  
}
```

Run:

```
gradlew taskTwo
```

Answer for Q4 – cleanTemp → buildTemp

```
tasks.register('cleanTemp') {  
    doLast {  
        println "Cleaning temp"  
    }  
}  
  
tasks.register('buildTemp') {  
    dependsOn 'cleanTemp'  
    doLast {  
        println "Building temp"  
    }  
}
```

Answer for Q5 – Order without dependency

```
tasks.register('alpha') {
    doLast {
        println "Alpha"
    }
}

tasks.register('beta') {
    mustRunAfter 'alpha'
    doLast {
        println "Beta"
    }
}
```

Answer for Q6 – Chain execution

```
tasks.register('prepare') {
    doLast {
        println "Prepare"
    }
}

tasks.register('compile') {
    dependsOn 'prepare'
    doLast {
        println "Compile"
    }
}

tasks.register('packageApp') {
    dependsOn 'compile'
    doLast {
        println "Package"
    }
}
```

Run:

```
gradlew packageApp
```

Answer for Q7 – Conditional execution via property

```
tasks.register('conditionalTask') {
    onlyIf {
        project.hasProperty('runTask')
    }
    doLast {
        println "Task Executed"
    }
}
```

Run:

```
gradlew conditionalTask -PrunTask
```

Answer for Q8 – UP-TO-DATE task

```
tasks.register('onceOnly') {
    outputs.file("$buildDir/once.txt")

    doLast {
        file("$buildDir/once.txt").text = "Executed"
        println "Executed"
    }
}
```

Answer for Q9 – Parent depends on child

```
tasks.register('childTask') {
    doLast {
        println "Child"
    }
}

tasks.register('parentTask') {
    dependsOn 'childTask'
    doLast {
        println "Parent"
    }
}
```

Answer for Q10 – Config vs Execution

```
tasks.register('configVsExec') {
    println "Config phase"
    doLast {
        println "Execution phase"
    }
}
```

Answer for Q11 – doFirst + doLast

```
tasks.register('dualAction') {
    doFirst {
        println "Start"
    }
    doLast {
        println "End"
    }
}
```

Answer for Q12 – File creator with inputs/outputs

```
tasks.register('fileCreator') {
    inputs.property("content", "hello")
    outputs.file("$buildDir/output.txt")

    doLast {
        file("$buildDir/output.txt").text = "hello"
        println "File created"
    }
}
```


Answer for Q13 – Skip on CI

```
tasks.register('skipOnCI') {
    onlyIf {
        System.getenv("CI") != "true"
    }
    doLast {
        println "Running locally"
    }
}
```

Answer for Q14 – Shared dependency

```
tasks.register('setup') {
    doLast {
        println "Setup"
    }
}

tasks.register('taskA') {
    dependsOn 'setup'
    doLast {
        println "Task A"
    }
}

tasks.register('taskB') {
    dependsOn 'setup'
    doLast {
        println "Task B"
    }
}
```

Answer for Q15 – Conditional based on task graph (Hard)

```
tasks.register('safeTask') {
    onlyIf {
        !gradle.taskGraph.hasTask(':dangerTask')
    }
    doLast {
        println "Safe execution"
    }
}

tasks.register('dangerTask') {
    doLast {
        println "Danger!"
    }
}
```

15. Tasks - Interview questions

BASIC QUESTIONS (Foundations)

1. What is a Gradle task?

Answer: A Gradle task is a **single unit of work** that Gradle performs, such as compiling code, running tests, or creating a JAR file. Every action in a Gradle build is done through tasks.

2. Why are tasks needed in a Java project?

Answer: Tasks break the build process into **small, manageable, repeatable steps**. They automate compilation, testing, packaging, and running the application, ensuring consistency and reducing manual effort.

3. What happens when you run `gradlew build`?

Answer: Gradle executes the `build` task, which depends on other tasks like `compileJava`, `test`, and `jar`. It compiles the code, runs tests, and packages the application.

4. What is the difference between `build` and `assemble` tasks?

Answer: `assemble` only builds the output artifacts (like JAR files), while `build` does everything `assemble` does **plus runs tests and checks**.

5. What does the `clean` task do?

Answer: The `clean` task deletes the `build/` directory, removing all compiled files and previous build outputs to start fresh.

6. Which task compiles Java source code in a Gradle project?

Answer: The `compileJava` task compiles Java source files from `src/main/java`.

7. How do you list all available tasks in a Gradle project?

Answer: By running: `gradlew tasks`

8. What is a lifecycle task? Give examples.

Answer: A lifecycle task groups other tasks but does not perform work itself.
Examples: `build`, `assemble`, `check`.

9. Where are compiled `.class` files stored after a build?

Answer: They are stored in: `build/classes/java/main`

10. How do you run a single task from the command line?

Answer: By running: `gradlew <taskName>`

Example: `gradlew test`

INTERMEDIATE QUESTIONS

11. What is task dependency? How does Gradle resolve execution order?

Answer: A task dependency means one task must run before another. Gradle builds a **task dependency graph** and automatically determines the correct execution order.

12. Difference between `dependsOn` and `mustRunAfter`?

Answer: `dependsOn` enforces execution and dependency, while `mustRunAfter` only controls order **if both tasks are executed**, without creating a dependency.

13. What is the difference between configuration phase and execution phase?

Answer: Configuration phase: Gradle reads build scripts and configures tasks.

- **Execution phase:** Gradle executes only the requested tasks.

14. Why should logic be inside `doLast {}` instead of directly in the task?

Answer: Code outside `doLast {}` runs during configuration, even if the task is not executed. `doLast {}` ensures logic runs **only when the task executes**.

15. How does Gradle decide which tasks to execute?

Answer: Gradle analyzes the requested task and its dependencies, builds a task graph, and executes only the required tasks.

16. What happens if two tasks depend on the same task?

Answer: Gradle executes the shared dependency **only once**, improving efficiency.

17. How can you create a custom task in Gradle?

Answer: By defining it in `build.gradle`, for example:

```
tasks.register('myTask') {
    doLast {
        println "Hello"
    }
}
```

18. What is incremental build and how do tasks support it?

Answer: Incremental build means Gradle skips tasks whose inputs and outputs haven't changed. Tasks support this by declaring inputs and outputs.

19. What is a task graph?

Answer: A task graph is a **dependency tree** showing which tasks need to run and in what order.

20. How does the `test` task know which tests to run?

Answer: It scans compiled test classes under `src/test/java` and runs methods annotated with `@Test`.

ADVANCED / IMPRESSION QUESTIONS

21. What is task avoidance and why is `tasks.register()` preferred?

Answer: Task avoidance means tasks are created only when needed. `tasks.register()` improves performance by avoiding unnecessary task configuration.

22. Difference between `tasks.create()` and `tasks.register()` ?

Answer: `tasks.create()` creates and configures tasks immediately, while `tasks.register()` delays creation until execution is required.

23. What are inputs and outputs of a task? Why are they important?

Answer: Inputs are files or values a task uses; outputs are the results it produces. They enable **incremental builds and caching**.

24. How does Gradle skip tasks using UP-TO-DATE checks?

Answer: Gradle compares task inputs and outputs. If nothing changed, the task is marked **UP-TO-DATE** and skipped.

25. How do Gradle tasks help in CI/CD pipelines (like Jenkins)?

Answer: CI tools trigger Gradle tasks (`build`, `test`, `check`) to ensure automated, repeatable, and consistent builds across environments.

More things to Learn

1. Explain Gradle vs Maven
2. Walk through a real Gradle project
3. Explain Spring Boot Gradle structure
4. Give you interview questions
5. Help you create your first Gradle project step-by-step
6. Add **external dependency** (example: Apache Commons)
7. Create **unit tests with JUnit**
8. Convert this to a **Spring Boot Gradle project**
9. Learn **multi-module Gradle project**